

Часть 2. ECS внутри: Components, Systems, Baker

В первой части — почему ECS быстрее под капотом: AoS vs SoA, GC, virtual dispatch.

Теперь — из чего этот стек реально состоит на уровне кода. Разбор сущностей и production-паттернов на примере Survival-фичи: top-down arena с волнами врагов, авто-стрельба, прокачка по уровням.

Entity — это просто ID

Entity в Unity ECS — это `struct { int Index; int Version; }`. Никаких полей, никакого поведения. Просто 64-битный идентификатор для поиска компонентов в массивах.

```
Entity playerEntity = SystemAPI.GetSingletonEntity<PlayerTag>();
```

Эта строка возвращает ID единственной entity в мире, на которой висит компонент `PlayerTag`. Дальше через этот ID можно получить любой её компонент: `SystemAPI.GetComponent<Health>(playerEntity)`.

Принять это переключение мышления — самый сложный момент в переходе с `MonoBehaviour`. Привычка: `player.Health.Value`. В ECS: entity находится по тегу, затем component достаётся из глобального массива по ID. **Объект как сущность исчез — остался ID, по которому ассемблятся данные.**

Компоненты: четыре типа

`IComponentData` — базовый интерфейс. Но на практике встречаются четыре разные роли компонентов. Понимание ролей — половина грамотной архитектуры ECS.

Tag — маркер архетипа

Zero-size struct. Без полей. Используется только для **фильтрации в Query**.

```
public struct PlayerTag : IComponentData {}  
public struct EnemyTag : IComponentData {}  
public struct DeadTag : IComponentData {}
```

В Query:

```
SystemAPI.Query<RefRW<LocalTransform>>()  
    .WithAll<EnemyTag>()  
    .WithNone<DeadTag>()
```

Под капотом каждая комбинация компонентов формирует **архетип**. `Entity { EnemyTag, Health, MoveSpeed }` и `Entity { EnemyTag, Health, MoveSpeed, DeadTag }` — это **два разных архетипа** и физически разные чанки памяти. Когда `DeadTag` навешивается на врага, entity мигрирует в другой чанк, и Query с `.WithNone<DeadTag>()` её больше не видит.

Это бесплатная фильтрация — без `if (enemy.isDead)` в каждом job'e.

Data — данные сущности

Обычный `IComponentData` с полями. **Одна семантическая роль = один компонент.**

```
public struct Health      : IComponentData { public float Value; }
public struct MoveSpeed   : IComponentData { public float Value; }
public struct AimDirection : IComponentData { public float3 Value; }
```

Анти-паттерн: запихнуть всё в одну struct по привычке OOP:

```
// НЕ ДЕЛАЙТЕ ТАК
public struct EnemyData : IComponentData {
    public float Health;
    public float Speed;
    public float Damage;
    public float Radius;
}
```

Расплата за такой подход приходит на параллельных job'ах. `EnemyMoveJob` получает `RefRW<EnemyData>` ради `Speed`, параллельно `DamageJob` хочет `Damage` тоже как `RefRW` — Unity safety-system закономерно ругается на конфликт. Получается искусственная сериализация того, что должно идти параллельно.

Когда каждое поле — отдельный компонент, эта проблема исчезает: один job берёт `RefRW<MoveSpeed>`, другой `RefRW<Damage>`, конкуренции нет.

Singleton — глобальное состояние

Тот же `IComponentData`, но **в единственном экземпляре** в world. Идиома, не язык.

```
public struct PlayerPosition : IComponentData { public float3 Value; }
```

Доступ через специальный API:

```
float3 pos = SystemAPI.GetSingleton<PlayerPosition>().Value;
SystemAPI.GetSingletonRW<PlayerPosition>().ValueRW.Value = newPos;
```

ECS не запрещает «глобальное состояние», но обязывает положить его на entity. Это даёт **чистый API**: любой singleton можно прочитать и записать из любой системы, без статиков и без managed-зависимостей.

Типичные singleton'ы в Survival-фиче: **PlayerPosition** (зеркало позиции игрока для камер и систем целеуказания), **Weapon** (настройки оружия), **SpawnConfig**, **DifficultyState**.

Buffer — динамический массив на entity

Когда нужен массив, привязанный к **одной** entity (не несколько entities одного типа), — используем **IBufferElementData**:

```
public struct DifficultyStage : IBufferElementData
{
    public float TimeThreshold;
    public float HpMultiplier;
    public float DamageMultiplier;
    public float SpawnIntervalMultiplier;
    public int CountPerWaveAddend;
}
```

Это **строка** массива. Сам массив (**DynamicBuffer<DifficultyStage>**) висит на entity и хранит curve роста сложности — например, 8 stages по времени игры.

Доступ:

```
DynamicBuffer<DifficultyStage> stages =
    SystemAPI.GetSingletonBuffer<DifficultyStage>(true);

for (int i = 0; i < stages.Length; i++) { /* ... */ }
```

Если бы stages было 500 — это всё равно работает: буферы рассчитаны на масштаб.

Authoring + Baker — мост из Editor в ECS

В Editor работают с GameObject + MonoBehaviour. ECS работает с Entity. Между ними — **Baker**: специальный конвертер, который запускается при импорте SubScene.

```
public class EnemyAuthoring : MonoBehaviour
{
    public float health = 30f;
    public float moveSpeed = 10f;
    public float contactDamagePerHit = 5f;
    public float contactRadius = 1.5f;

    public class Baker : Baker<EnemyAuthoring>
```

```

{
    public override void Bake(EnemyAuthoring authoring)
    {
        Entity entity = GetEntity(TransformUsageFlags.Dynamic);

        AddComponent(entity, new EnemyTag());
        AddComponent(entity, new Health { Value = authoring.health });
        AddComponent(entity, new MoveSpeed { Value = authoring.moveSpeed });
        AddComponent(entity, new ContactDamage
        {
            DamagePerHit = authoring.contactDamagePerHit,
            Radius = authoring.contactRadius
        });
    }
}

```

Что здесь происходит:

1. **EnemyAuthoring** — обычный MonoBehaviour на префабе. Дизайнер крутит ползунки в инспекторе.
2. При импорте SubScene Unity запускает **Bake()**. Метод возвращается с готовой Entity и набором компонентов.
3. Результат сохраняется в **бинарь SubScene'a**. В runtime это уже не MonoBehaviour, а голые ECS-данные.

TransformUsageFlags.Dynamic — флаг говорит: «эта entity нужна с Transform-компонентами (**LocalTransform**, **LocalToWorld**), её будут двигать». Без флага entity получится без Transform — и движение через **LocalTransform.Position += ...** не сработает.

Подводный камень: добавил поле в Authoring → нужно реимпортировать SubScene. Bake-результат закеширован в бинаре. Без reimport'a поле в Editor видно, но в runtime его нет — типичный источник часов отладки «почему компонент пустой». ПКМ на SubScene → Reimport.

System (ISystem) — где живёт логика

Современный API — **ISystem** (struct-based). Старый **SystemBase** (class-based, managed) тоже работает, но в новом коде рекомендуется **ISystem** — он Burst-friendly и не несёт managed-overhead.

```

[UpdateInGroup(typeof(SimulationSystemGroup))]
[UpdateBefore(typeof(EnemySpawnSystem))]
[BurstCompile]
public partial struct DifficultyProgressionSystem : ISystem
{
    public void OnCreate(ref SystemState state)
    {
        state.RequireForUpdate<DifficultyState>();
    }
}

```

```

[BurstCompile]
public void OnUpdate(ref SystemState state)
{
    var diffRW = SystemAPI.GetSingletonRW<DifficultyState>();
    ref var diff = ref diffRW.ValueRW;
    diff.ElapsedTime += SystemAPI.Time.DeltaTime;

    var stages = SystemAPI.GetSingletonBuffer<DifficultyStage>(true);
    if (stages.Length == 0) return;

    int target = diff.CurrentStageIndex;
    while (target + 1 < stages.Length
        && stages[target + 1].TimeThreshold <= diff.ElapsedTime)
        target++;

    if (target != diff.CurrentStageIndex && target >= 0)
    {
        diff.CurrentStageIndex = target;
        diff.HpMultiplier = stages[target].HpMultiplier;
        diff.SpawnIntervalMultiplier = stages[target].SpawnIntervalMultiplier;
        diff.CountPerWaveAddend = stages[target].CountPerWaveAddend;
    }
}
}

```

Структура любого ISystem:

- **OnCreate** — один раз при инициализации. Здесь же **RequireForUpdate<T>()**, который **гасит OnUpdate** пока в мире нет компонента указанного типа. Защита от exception'ов на старте до bake'a SubScene.
- **OnUpdate** — каждый кадр. Чисто данные, чистые операции, никаких managed-вызовов внутри (если **[BurstCompile]**).
- **OnDestroy** — один раз при shutdown. На практике используется редко.

partial struct — обязательно, source generator досыпает скрытый код в parallel-file.

SystemAPI шпаргалка

Что нужно	API
Один компонент с уникальной entity	<code>SystemAPI.GetSingleton<T>()</code>
Запись в singleton	<code>SystemAPI.GetSingletonRW<T>().ValueRW</code>
Компонент конкретной entity	<code>SystemAPI.GetComponent<T>(entity)</code>
Buffer на singleton	<code>SystemAPI.GetSingletonBuffer<T>(readOnly)</code>
Перебор entities с фильтром	<code>SystemAPI.Query<RefRO<T>, RefRW<U>>().WithAll<>().WithNone<>()</code>

RefRO = read-only, **RefRW** = read-write. Это подсказки **safety-system'е** для проверки конфликтов между job'ами — Unity автоматически вычисляет какие job'ы могут идти параллельно, а какие должны быть сериализованы.

EntityCommandBuffer — structural changes безопасно

Внутри job (или вообще когда параллелим) **нельзя** делать `EntityManager.AddComponent(...)` или `Instantiate(...)`. Эти операции мутируют архетипы — на ходу пересортируют entities между чанками. Делать это пока другой job читает чанк = crash.

Решение — **EntityCommandBuffer (ECB)**. Буфер команд, которые накопились за кадр и отыгрываются в **фиксированной точке** жизненного цикла.

Два главных вида:

ECB system	Когда playback	Куда использовать
<code>BeginSimulationEntityCommandBufferSystem</code>	В начале следующего sim-tick	Instantiate новых entity — появятся сразу
<code>EndSimulationEntityCommandBufferSystem</code>	В конце текущего sim-tick	DestroyEntity / <code>AddComponent<DeadTag></code> — entity доживёт до конца кадра

Spawn через BeginSim:

```
public void OnUpdate(ref SystemState state)
{
    var ecb =
    SystemAPI.GetSingleton<BeginSimulationEntityCommandBufferSystem.Singleton>()
        .CreateCommandBuffer(state.WorldUnmanaged);

    for (int i = 0; i < count; i++)
    {
        Entity enemy = ecb.Instantiate(prefab);
        ecb.SetComponent(enemy, new LocalTransform { Position = spawnPos });
        ecb.SetComponent(enemy, new Health { Value = baseHp * difficultyMultiplier
    });
    }
}
```

Типичная ловушка: использовать `EndSimulation` ECB для спавна XP-гема в момент убийства врага. Между `Instantiate` и `SetComponent(LocalTransform)` система рендеринга успевает отрисовать **один кадр гема в позиции (0, 0, 0)** — посередине арены вспыхивает точка. Лечится переходом на **BeginSimulation** ECB: новая entity и её LocalTransform появляются в одном sim-tick.

Pattern: DeadTag — отложенное уничтожение

Анти-паттерн: пометил врага мёртвым → сразу `EntityManager.DestroyEntity(enemy)`. Пока кадр не закончен, другие системы могут на эту entity ссылаться (например, `DamageSystem` уже передал её `Entity` в job).

Идиома: **mark with `DeadTag` → cleanup в одной отдельной системе в конце кадра.**

```
// В DamageJob, PickupXpJob, LifetimeSystem — любой кто решает "пора умереть":
Ecb.AddComponent<DeadTag>(entity);

// Отдельная система в конце SimulationSystemGroup:
public partial struct DestroyDeadSystem : ISystem
{
    public void OnCreate(ref SystemState state) => state.RequireForUpdate<DeadTag>
    ();

    public void OnUpdate(ref SystemState state)
    {
        var ecb =
        SystemAPI.GetSingleton<EndSimulationEntityCommandBufferSystem.Singleton>()
            .CreateCommandBuffer(state.WorldUnmanaged);

        foreach (var (_, entity) in
            SystemAPI.Query<RefRO<DeadTag>>().WithEntityAccess())
        {
            ecb.DestroyEntity(entity);
        }
    }
}
```

Что даёт этот паттерн:

- **Одна точка destroy** — нет дублирующей логики по проекту.
- **Death-effects можно навешивать** до фактического destroy: `DamageSystem` ставит `DeadTag` и **в этом же месте** спавнит XP-гем на позиции смерти. Если бы destroy происходил сразу, позиция entity была бы уже потеряна.
- **Безопасные параллельные jobs**: job ставит `DeadTag` через ECB, structural change не происходит сразу.

Pattern: Event entities — сигналы между ECS и Mono

Когда нужно отправить **сигнал** (а не значение) — между системами или из ECS-мира в `MonoBehaviour`, — создаётся временная entity с одним event-компонентом. Потребитель её прочитает и удалит.

```
public struct EnemyAttackEvent : IComponentData
{
    public Entity Attacker;
}
```

Эмит в `EnemyContactDamageJob` (когда зомби наносит удар):

```
Entity eventEntity = Ecb.CreateEntity();
Ecb.AddComponent(eventEntity, new EnemyAttackEvent { Attacker = entity });
```

Потребление в `EnemyVisualPoolManager` (Mono, на стороне презентации):

```
using var events = _attackEventsQuery
    .ToComponentDataArray<EnemyAttackEvent>(Allocator.Temp);
using var entities = _attackEventsQuery
    .ToEntityArray(Allocator.Temp);

for (int i = 0; i < events.Length; i++)
{
    if (_active.TryGetValue(events[i].Attacker, out var visual))
        visual.Animator.SetTrigger("Attack");
}
em.DestroyEntity(entities);
```

В типичной Survival-фиче тот же паттерн используется неоднократно:

- `DamageEvent` → Mono-виджет рисует floating «-5» поверх врага
- `EnemyAttackEvent` → Animator зомби играет Attack
- `PlayerShotEvent` → Animator игрока играет натяжение лука

Почему не статический event-bus с callback'ами? Потому что callback — managed delegate, он не работает в Burst-job'e. А события логически возникают именно в Burst-job'e, где принимается решение о damage. Entity как event переживает мир Burst и хорошо ложится в архитектуру: эмит — это просто `Ecb.CreateEntity()`, нативная операция без managed-обёрток.

Что дальше

Часть 3 — **Jobs**. Как параллелить job'ы на `ScheduleParallel`, как избежать race conditions с `ComponentLookup`, и почему **single-thread иногда оказывается быстрее multi-thread** — с разбором реального примера где так и оказалось.

Связанная литература

- [Unity Entities package documentation](#)
- Часть 1 серии — «ECS DOTS: когда стандартный Unity сдаётся»